

ROS 2 Path Planner & Controller Master Benchmarking & Architecture Guide

This document is the unified, comprehensive master guide for the ROS 2 Path Planning & Control Benchmarking Suite, designed specifically for the **ERC (European Rover Challenge)** autonomous rover. It combines usage manuals, system architecture explanations, mathematical metric derivations, and map customizer guides into a single authoritative reference.

1. Quick Start: One-Click Automated Execution

You can build, source, launch tests against your **erc_path_planner** (Smac Hybrid A* + MPPI Controller), and view reports using the automated test script:

```
# Execute from the project root (/home/saif/Desktop/MESEKET/Autonomous-27)
./Autonomous_Ws/testing/PathPlanner/run_testing_suite.sh
```

To run a single targeted scenario (e.g. **scattered_rocks_detour**):

```
./Autonomous_Ws/testing/PathPlanner/run_testing_suite.sh scattered_rocks_detour
```

2. ■ ROS 2 Version Compatibility (Jazzy & Humble)

This testing suite is **100% dual-compatible** with both ROS 2 Jazzy and ROS 2 Humble.

- The core Python testing architecture (**rc_lpy**) uses standard, version-agnostic interfaces for **nav_msgs**, **geometry_msgs**, and ROS 2 topic lifecycle management.
- It runs seamlessly on ROS 2 Jazzy and ROS 2 Humble without requiring any code edits or conversions.

3. Node Architecture & Standalone Operation

The benchmarking suite operates as a black-box observer (**testing_node.py** and **benchmarking_node.py**) that interacts through standard ROS 2 interfaces:

Interface Topics:

- **/test_name** (**std_msgs/msg/String**): Sets the current test context (e.g., scenario name). Resets dynamic tracking states when switched. Defaults to **"general"**.
- **/global_costmap/costmap** & **/map** (**nav_msgs/msg/OccupancyGrid**): Provides obstacle and terrain cost information.
- **/start_pose** (**geometry_msgs/msg/PoseStamped**): The initial position of the robot.
- **/goal_pose** (**geometry_msgs/msg/PoseStamped**): The target destination position.
- **/plan** (**nav_msgs/msg/Path**): The global route calculated by Smac Hybrid A*.
- **/cmd_vel** (**geometry_msgs/msg/Twist**): The motor steering and throttle commands output by MPPI Controller.
- **/planner_internal_time** (**std_msgs/msg/Float32**): The planner's internal execution duration.

Execution Flow:

1. Scenario Orchestration (**testing_node**):

- Launches **erc_path_planner**'s bringup (**ros2 launch erc_path_planner path_planning.launch.py**).
- Converts PNG map images into ROS 2 **OccupancyGrid** messages and publishes them to **/map** and **/global_costmap/costmap**.

- Publishes static coordinate transforms (`map -> odom -> base_link`) for the rover's starting position.
- Publishes the target destination on `/goal_pose`.

2. Metrics Evaluation (`benchmarking_node`):

- Intercepts global paths (`/plan`) and motor commands (`/cmd_vel`).
- Calculates path length, planning latency, footprint collision count against obstacle cells, safety distance margins, turning angles, and curvature radii.
- Dynamically injects mid-run dynamic obstacles to test real-time MPPI/Smac replanning.
- Computes a weighted **Path Planning Score** out of 100 points (Passing threshold: ≥ 85.0).

3. Report Compilation (`report_generator.py`):

- Generates visual path comparison plots (PNG), raw JSON/CSV data, an interactive web dashboard (`reports/numerical_report.html`), and a printable PDF report (`reports/numerical_report.pdf`).

4. Mathematical Formulations for Path & Control Metrics

The benchmarking monitor calculates several geometric, physical, and cost characteristics:

A. Path Distance & Euclidean Ratio

Let the planned path consist of coordinates $P = [P_0, P_1, \dots, P_n]$, where each $P_i = (x_i, y_i)$.

- **Distance Covered (L):** The total length of the path:

$$L = \sum_{i=0}^{n-1} \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}$$

- **Euclidean Distance (D_{euc}):** The straight-line distance between start pose P_{start} and goal pose P_{goal} :

$$D_{\text{euc}} = \sqrt{(x_{\text{goal}} - x_{\text{start}})^2 + (y_{\text{goal}} - y_{\text{start}})^2}$$

- **Path Length Ratio:**

$$\text{Ratio} = \frac{L}{D_{\text{euc}}} \quad (\text{Ratio} = 1.0 \text{ if } D_{\text{euc}} = 0.0)$$

B. Turn Angles (Degrees)

For each three consecutive waypoints P_{i-1} , P_i , and P_{i+1} , the deviation angle θ_i represents the change in heading direction:

1. Form vectors $\vec{v}_1 = P_i - P_{i-1}$ and $\vec{v}_2 = P_{i+1} - P_i$.

2. Calculate magnitudes $|\vec{v}_1|$ and $|\vec{v}_2|$.

3. Compute the angle of deflection:

$$\theta_i = \arccos\left(\frac{|\vec{v}_1| \cdot |\vec{v}_2|}{|\vec{v}_1| |\vec{v}_2|}\right) \quad (-1.0, 1.0) \times \frac{180}{\pi}$$

4. The system records the **Minimum**, **Maximum**, and **Average** turn angles along the path.

C. Turn Radii (Meters)

The turn radius R_i at a waypoint P_i is computed as the circumcircle radius of the triangle formed by P_{i-1} , P_i , and P_{i+1} :

1. Let the side lengths of triangle $\Delta P_{i-1}P_iP_{i+1}$ be $a = |P_{i+1} - P_i|$, $b = |P_{i+1} - P_{i-1}|$, and $c = |P_i - P_{i-1}|$.

2. Compute the area of the triangle using the determinant cross-product:

$$\text{Area}_{\Delta} = \frac{1}{2} \left| x_{i-1}(y_i - y_{i+1}) + x_i(y_{i+1} - y_{i-1}) + x_{i+1}(y_{i-1} - y_i) \right|$$

3. Compute circumradius:

$$R_i = \frac{a \cdot b \cdot c}{4 \cdot \text{Area}_{\Delta}}$$

4. **Collinear Filtering:** If the three points are collinear, $\text{Area}_{\Delta} = 0$. To ensure physical significance for robot kinematics:

- R_i is only computed if $\theta_i > 1.0^\circ$.
- Any $R_i > 100.0$ meters is capped at 100.0 m.

5. The system reports the **Minimum**, **Maximum**, and **Average** turn radii.

D. Nearest Obstacle Distance (Clearance)

For a costmap with origin (x_o, y_o) , resolution r , and a 2D occupancy grid where cell values ≥ 100 indicate obstacles:

1. Identify all obstacle cell coordinates $O_j = (x_o + \text{col} \cdot r, y_o + \text{row} \cdot r)$.
2. Compute the minimum Euclidean distance from each path point P_i to obstacle points O_j :

$$\text{Clearance} = \min_{i,j} \sqrt{(x_i - x_{O_j})^2 + (y_i - y_{O_j})^2}$$

E. Path Cost

Path cost evaluates terrain steepness:

1. For each waypoint P_i , find costmap cell $C_i = \text{cost_grid}[\text{row}, \text{col}]$.
2. **Total Path Cost:** $C_{\text{total}} = \sum_{i=0}^n C_i$
3. **Average Path Cost:** $C_{\text{avg}} = \frac{C_{\text{total}}}{n+1}$

5. Report Styling & Visual Design System

The visual design is implemented in [report_generator.py](#) and produces PDF/HTML reports mirroring professional engineering diagnostics:

- **Color Palette:** Premium dark blue (**#1A365D**) for table headers, slate gray (**#718096**) for labels, green (**#C6F6D5** bg / **#22543D** text) for success, and red (**#FED7D7** bg / **#9B2C2C** text) for failures.
- **Header & Footer:** Features uppercase title headers, dividing rules, generation timestamp, and dynamic page numbering ("Page X of Y").
- **Top Evaluation Banner:** A status box displaying whether overall benchmark pass rate ($\geq 85\%$) was achieved.
- **Summary Cards:** 4-column flex grid detailing Path Success Rate, Mean Planning Time, Max Turn Angle, and Min Obstacle Clearance.
- **Visual Path Comparison Plots:** Generates high-resolution PNG plots showing **Reference Path (Green)** vs. **Your Rover Path (Red)** with footprint safety circles.

6. What To Expect (Outputs)

Terminal Summary Output:

```
===== SCENARIO RESULTS: marsyard_labyrinth =====
Status:          SUCCESS
Planning Time:    0.450 s (Score: 100.0/100)
Path Length:      12.40 m (Ratio: 1.15, Score: 95.0/100)
Turn Angles:      Min: 2.1° | Max: 45.0° | Avg: 12.3°
Blocked Cells:    0 (Score: 100.0/100)
Average Path Cost: 15.2 (Score: 84.8/100)
Safety Margin:    0.42 m
Replanning:       Score: 100.0/100 (Time: 0.320 s)
-----
PATH PLANNING SCORE: 94.50 / 100
OUTCOME:          PASS (Required: >= 85.0/100)
=====
```

Generated Files in `reports/`:

- `numerical_report.html` – Interactive web browser dashboard.
- `numerical_report.pdf` – Printable PDF report with tables and plots.
- `result_scenario.png` – High-res visual path plots.

7. Manual Build & Execution

Always run `colcon build` from the project root (`/home/saif/Desktop/MESEKET/Autonomous-27`):

```
# 1. Navigate to project root
cd /home/saif/Desktop/MESEKET/Autonomous-27

# 2. Build packages
colcon build --packages-select global_path_benchmarking erc_path_planner terrain_geometry_msgs

# 3. Source environment
source install/setup.bash

# 4. Launch benchmark
ros2 launch global_path_benchmarking benchmark.launch.py clean:=true
```

8. How to Add Custom Maps & Test Scenarios

Step 1: Create a Map PNG Image

Add a PNG image inside `maps/` (recommended size **200x200** pixels):

- **White pixels (255)**: Free space.
- **Black pixels (0)**: Hard walls / rocks.
- **Gray pixels (1–254)**: Rough terrain / slopes.

Step 2: Register Scenario in `config/scenarios.yaml`

```

scenarios:
  - id: "custom_marsyard_arena"
    map_image: "maps/custom_marsyard_arena.png"
    resolution: 0.05
    origin: [-5.0, -5.0]
    robot_radius: 0.35

    start: [-4.0, -4.0]
    goal: [4.0, 4.0]

    reference_path:
      - [-4.0, -4.0]
      - [0.0, 0.0]
      - [4.0, 4.0]

    dynamic_obstacles:
      - trigger_time: 0.5
        x: 0.0
        y: 0.0
        radius: 0.4

```

Step 3: Verify and Run

```

# Verification plot
ros2 launch global_path_benchmarking benchmark.launch.py verify:=true

# Benchmark run
ros2 launch global_path_benchmarking benchmark.launch.py scenario_id:=custom_marsyard_arena clean:=true

```

9. ■ Directory Structure

```

testing/PathPlanner/
■■■■ run_testing_suite.sh           # Master one-click automated test runner
■■■■ README.md                     # Unified master instruction & architecture manual
■■■■ package.xml                   # ROS 2 package manifest
■■■■ setup.py                      # Python package build script
■■■■ setup.cfg
■■■■ ROS2_Path_Planning_Benchmarking_Manual.pdf
■■■■ PathPlanner_guide.pdf         # Auto-generated PDF Guide
■■■■ config/
■   ■■■■ scenarios.yaml            # Scenario definitions & coordinates
■   ■■■■ benchmark_config.yaml    # Evaluation weights & limits
■■■■ launch/
■   ■■■■ benchmark.launch.py      # Main launch file
■■■■ maps/                        # PNG map images
■■■■ reports/                     # Output HTML, PDF & PNG plots
■■■■ global_path_benchmarking/    # Python node scripts
■■■■   __init__.py
■■■■   testing_node.py
■■■■   benchmarking_node.py
■■■■   report_generator.py

```

10. ■ Benchmark Evaluation Matrix

$$\text{Planning Score} = 0.25 S_{\text{success}} + 0.15 S_{\text{time}} + 0.25 S_{\text{obstacle}} + 0.15 S_{\text{cost}} + 0.10 S_{\text{length}} + 0.10 S_{\text{replan}}$$

| Sub-score | Weight | Raw Target | Formula / Condition |

| :--- | :--- | :--- | :--- |

| **Planning Success (\$S_{success}\$)** | 25% | Path found | **100** if goal reached, **0** if failed/timeout. |

| **Planning Time (\$S_{time}\$)** | 15% | $\leq 2.0\text{s}$ | **100** if $T \leq 2\text{s}$, **0** if $T \geq 10\text{s}$, else linear interpolation. |

| **Obstacle Avoidance (\$S_{obstacle}\$)** | 25% | \$0\$ collisions | **100** if footprint never touches lethal cells (≥ 100), else **0**. |

| **Path Cost (\$S_{cost}\$)** | 15% | Min slope cost | $100 - C_{avg}$ mean cost traversed. |

| **Path Length Ratio (\$S_{length}\$)** | 10% | ≤ 1.35 | **100** if ratio ≤ 1.0 , **0** if ratio ≥ 1.35 , else linear interpolation. |

| **Replanning (\$S_{replan}\$)** | 10% | $\leq 2.0\text{s}$ detour | **100** if detour successful under 2.0s, else **0**. |